Tutorial de Integração PS–PL na Kria KV260 Utilizando PYNQ e Acelerador HOG via AXI-DMA

Nathan Spínola Zeidan
180025864

2025.2

Sumário

1 Arquitetura Geral do Sistema **2**

2 Fundamentos da Comunicação PS–PL **3**
2.1 AXI-Lite: Controle do Acelerador 3
2.2 AXI4-Stream: Transporte de Dados 4
2.3 AXI-DMA: Ponte entre Memória e AXI-Stream 4

3 Overlay no Framework PYNQ **5**

4 Aplicação do Framework PYNQ: Passo a Passo **6**
4.1 Passo 1: Carregamento do Overlay 6
4.2 Passo 2: Acesso aos IPs e Interfaces 6
4.3 Passo 3: Leitura de Parâmetros do Hardware 6
4.4 Passo 4: Alocação de Buffers para o DMA 7
4.5 Passo 5: Preparação da ROI e Formatação do Vetor de Entrada 7
4.6 Passo 6: Configuração do IP via AXI-Lite (Start e Clear) 8
4.7 Passo 7: Transferência de Dados com AXI-DMA (MM2S e S2MM) 9
4.8 Passo 8: Leitura do Descritor HOG e Função de Execução Completa 9

5 Conclusão **10**

1 Arquitetura Geral do Sistema

A plataforma Kria KV260 é baseada em um SoC da família Zynq UltraScale+, o que significa que processadores e FPGA coexistem dentro do mesmo encapsulamento físico. Diferentemente de uma solução tradicional composta por CPU e FPGA separados, a arquitetura Zynq permite que ambos compartilhem memória e se comuniquem por meio de interconexões internas.

O dispositivo integra dois domínios principais:

- **Processing System (PS):** conjunto de processadores ARM responsáveis pela execução do sistema operacional Linux e das aplicações em alto nível;
- **Programmable Logic (PL):** FPGA onde o acelerador HOG foi implementado em RTL.

Neste projeto, o PS executa código Python por meio do framework PYNQ. O papel do PS não é realizar o cálculo do descritor HOG, mas sim orquestrar o processamento. Especificamente, ele é responsável por:

- Preparar a Região de Interesse (ROI);
- Organizar os dados na memória;
- Enviar os pixels ao hardware;
- Receber o descritor HOG processado;
- Realizar validações e medições de desempenho.

Por sua vez, o PL contém a lógica dedicada ao processamento intensivo. Nesse domínio encontram-se:

- O IP do descritor HOG implementado em hardware;
- Uma interface AXI-Lite para controle e leitura de registradores;
- Interfaces AXI4-Stream para entrada e saída de dados;
- Um controlador AXI-DMA responsável por movimentar dados entre memória e stream.

A comunicação entre PS e PL ocorre conforme o seguinte fluxo lógico:

$$\text{PS (memória)} \rightarrow \text{AXI-DMA (MM2S)} \rightarrow \text{AXI4-Stream} \rightarrow \text{HOG (PL)} \rightarrow \text{AXI4-Stream} \rightarrow \text{AXI-DMA (S2MM)} \rightarrow \text{PS (memória)}$$

Esse fluxo ilustra um ponto fundamental: o PS nunca escreve diretamente no IP HOG pixel a pixel. Em vez disso, ele delega a movimentação de dados ao controlador DMA, que atua como intermediário eficiente entre memória e hardware.

2 Fundamentos da Comunicação PS–PL

A comunicação entre o Processing System (PS) e a Programmable Logic (PL) na arquitetura Zynq UltraScale+ ocorre por meio do protocolo AXI (Advanced eXtensible Interface). O AXI é parte da especificação AMBA da ARM e foi projetado para permitir comunicação modular, escalável e de alta performance entre blocos de hardware.

Neste trabalho foram usadas três variações do AXI:

- AXI-Lite (controle);
- AXI4-Stream (fluxo contínuo de dados);
- AXI-DMA (ponte entre memória e stream).

Cada uma dessas interfaces possui uma função específica dentro da arquitetura do sistema.

2.1 AXI-Lite: Controle do Acelerador

AXI-Lite é uma versão simplificada do protocolo AXI, projetada especificamente para acesso a registradores de controle. Diferentemente das versões completas do AXI, ela não é otimizada para transferência massiva de dados, mas sim para leitura e escrita pontual de parâmetros.

No acelerador HOG utilizado neste projeto, a interface AXI-Lite permite:

- Acionar o processamento (registrador `start`);
- Limpar estados internos (registrador `clear`);
- Ler o número total de pixels esperados na entrada;
- Ler o número de palavras do descritor gerado na saída.

Do ponto de vista do software, o acesso ocorre como se estivéssemos manipulando posições de memória mapeadas:

```
def rd(addr):
    return hog.read(addr)

def wr(addr, value):
    hog.write(addr, value)
```

Essa abordagem é conhecida como *memory-mapped I/O*. O PS escreve em um endereço e, internamente, esse acesso é convertido em sinais no barramento AXI-Lite, que por sua vez atualizam registradores dentro do IP.

Em termos práticos, AXI-Lite funciona como o "painel de controle" do acelerador.

2.2 AXI4-Stream: Transporte de Dados

Enquanto o AXI-Lite é voltado ao controle, o AXI4-Stream é projetado para transferência eficiente de grandes volumes de dados. Ele opera como um canal unidirecional de fluxo contínuo, sem endereçamento explícito de memória.

No projeto desenvolvido:

- A ROI (pixels) é transmitida ao IP HOG por meio de um canal AXI4-Stream de entrada;
- O descritor HOG é retornado ao PS por meio de um canal AXI4-Stream de saída.

O protocolo AXI4-Stream baseia-se em um mecanismo de handshake síncrono, composto pelos sinais:

- `TVALID`: indica que o transmissor possui dado válido;
- `TREADY`: indica que o receptor está pronto para receber;
- `TDATA`: contém o dado transmitido;
- `TLAST`: indica o fim do pacote de dados.

A transferência de um dado ocorre somente quando `TVALID` e `TREADY` estão simultaneamente ativos. Isso garante que o produtor e o consumidor estejam sincronizados, evitando perda ou sobreposição de dados.

O sinal `TLAST` é particularmente importante neste projeto, pois ele indica o fim da ROI enviada ou o fim do descritor retornado. Caso o framing esteja incorreto, o sistema pode travar aguardando dados adicionais.

2.3 AXI-DMA: Ponte entre Memória e AXI-Stream

O AXI-DMA é o componente que conecta dois mundos distintos: o espaço de memória do PS e o domínio AXI4-Stream da PL.

Ele opera em dois modos complementares:

- **MM2S (Memory-Mapped to Stream):** envia dados da memória do PS para o AXI4-Stream;
- **S2MM (Stream to Memory-Mapped):** recebe dados do AXI4-Stream e os grava na memória.

No framework PYNQ, o uso do DMA é realizado por meio de dois canais:

```
dma.sendchannel    # MM2S
dma.recvchannel    # S2MM
```

O fluxo típico de execução do acelerador é:

1. Preparar o buffer de entrada na memória;
2. Configurar o IP via AXI-Lite;

3. Iniciar a recepção (S2MM);
4. Iniciar o envio (MM2S);
5. Aguardar a conclusão das transferências.

Iniciar primeiro o canal de recepção reduz o risco de deadlock, pois garante que o sistema esteja pronto para capturar o resultado assim que o hardware finalizar o processamento.

Em termos conceituais, o DMA funciona como um "motor de cópia inteligente", capaz de transferir grandes volumes de dados sem intervenção contínua da CPU, liberando o processador para executar outras tarefas.

3 Overlay no Framework PYNQ

No contexto do PYNQ, o termo *overlay* refere-se a um conjunto de arquivos que descrevem e configuram a lógica programável (PL) do dispositivo. Em termos práticos, o overlay é a representação carregável do projeto de hardware desenvolvido no Vivado.

Um overlay é composto, principalmente, por dois arquivos:

- **.bit**: arquivo de bitstream responsável por configurar fisicamente a FPGA;
- **.hwh**: arquivo de descrição de hardware, utilizado pelo PYNQ para identificar IPs, registradores e conexões AXI.

Quando o overlay é carregado pelo PYNQ, o bitstream configura a PL e o arquivo `.hwh` permite que o software descubra automaticamente:

- Quais IPs estão presentes;
- Quais interfaces AXI estão disponíveis;
- Quais registradores podem ser acessados;
- Como os blocos estão conectados.

Dessa forma, o desenvolvedor não precisa manipular diretamente endereços físicos complexos. O framework abstrai essa camada, permitindo que o hardware seja acessado por meio de objetos Python.

Neste projeto, o overlay contém:

- O IP customizado do descritor HOG;
- Uma interface AXI-Lite para controle;
- Interfaces AXI4-Stream para entrada e saída de dados;
- Um controlador AXI-DMA conectado ao HOG.

A seguir, será apresentado o passo a passo de como esse overlay foi utilizado no PYNQ para executar o acelerador HOG.

4 Aplicação do Framework PYNQ: Passo a Passo

Nesta seção será apresentado o procedimento utilizado para executar o acelerador HOG na plataforma Kria KV260 por meio do PYNQ. O objetivo é demonstrar claramente cada etapa da integração entre software e hardware.

4.1 Passo 1: Carregamento do Overlay

O primeiro passo consiste em carregar o overlay na lógica programável. Esse processo configura a FPGA com o bitstream previamente gerado no Vivado.

No PYNQ, isso é feito da seguinte forma:

```
from pynq import Overlay

BIT_PATH = "/root/jupyter_notebooks/Tracking_Nathan_32_2/hog.bit"
ol = Overlay(BIT_PATH)
```

Ao executar esse comando, o PYNQ realiza duas ações principais:

1. Programa a PL com o arquivo `hog.bit`;
2. Lê o arquivo `hog.hwh` correspondente e cria objetos Python representando os IPs presentes no projeto.

Após o carregamento, é possível inspecionar os IPs disponíveis:

```
ol.ip_dict.keys()
```

Entre os IPs esperados estão o controlador AXI-DMA e o IP do descritor HOG.

4.2 Passo 2: Acesso aos IPs e Interfaces

Após o carregamento do overlay, os IPs podem ser acessados diretamente como atributos do objeto `Overlay`.

No projeto desenvolvido:

```
dma = ol.axi_dma_0
hog = ol.hog_ip_axi_Nathan_0
```

O objeto `dma` será utilizado para transferência de dados entre memória e stream, enquanto o objeto `hog` será utilizado para leitura e escrita de registradores via AXI-Lite.

4.3 Passo 3: Leitura de Parâmetros do Hardware

O IP HOG disponibiliza registradores que informam:

- O número total de pixels esperados na entrada;
- O número de palavras geradas na saída.

Esses valores são obtidos da seguinte forma:

```
TOTAL_PIXELS_IN = hog.read(0x10)
NUM_WORDS = hog.read(0x14)
```

A leitura desses registradores garante que o software utilize exatamente o mesmo tamanho configurado no hardware, evitando inconsistências e possíveis travamentos do sistema.

4.4 Passo 4: Alocação de Buffers para o DMA

Após identificar o tamanho de entrada e saída esperados pelo hardware, o próximo passo é criar os buffers que serão utilizados pelo AXI-DMA.

O controlador AXI-DMA realiza acesso direto à memória física do sistema. Para que essa operação funcione corretamente, o buffer precisa atender dois requisitos fundamentais:

- Estar localizado em uma região de memória fisicamente contígua;
- Ser acessível diretamente pelo hardware, sem realocação ou cópia intermediária.

A função `allocate()` do PYNQ cria buffers em memória contígua fisicamente e compatível com o controlador DMA. Isso garante que o hardware consiga ler e escrever nesses buffers diretamente.

A alocação é realizada da seguinte forma:

```
from pynq import allocate
import numpy as np

in_buf  = allocate(shape=(TOTAL_PIXELS_IN,), dtype=np.uint8)
out_buf = allocate(shape=(NUM_WORDS,), dtype=np.uint32)
```

Neste projeto:

- `in_buf` armazenará os pixels da ROI (8 bits por pixel);
- `out_buf` armazenará o descritor HOG retornado pelo hardware (palavras de 32 bits).

Caso buffers não compatíveis com o DMA sejam utilizados, o sistema pode apresentar travamentos, falhas silenciosas ou comportamento imprevisível.

4.5 Passo 5: Preparação da ROI e Formatação do Vetor de Entrada

O acelerador HOG implementado em hardware não recebe uma imagem bidimensional diretamente. Em vez disso, ele consome um fluxo linear de pixels via AXI4-Stream. Portanto, a ROI (matriz 2D) precisa ser convertida para um vetor 1D no formato esperado.

Neste projeto, a ROI é definida como uma janela quadrada de tamanho `ROI_SIZE` e, em seguida, é aplicado *padding* com zeros de largura `PADDING`. Essa etapa é importante porque o hardware foi projetado para operar sobre uma imagem já preenchida nas bordas.

O procedimento em software segue duas etapas:

1. Recortar uma ROI quadrada a partir da imagem original (em tons de cinza);
2. Aplicar o padding e converter a matriz final para vetor 1D.

O código utilizado é apresentado a seguir.

```
def crop_square_roi(img_gray, cx, cy, roi_size):
    H, W = img_gray.shape
    half = roi_size // 2
    x0 = int(round(cx)) - half
    y0 = int(round(cy)) - half

    roi = np.zeros((roi_size, roi_size), dtype=np.uint8)

    xs = max(0, x0)
    ys = max(0, y0)
    xe = min(W, x0 + roi_size)
    ye = min(H, y0 + roi_size)

    rx0 = xs - x0
    ry0 = ys - y0
    rx1 = rx0 + (xe - xs)
    ry1 = ry0 + (ye - ys)

    roi[ry0:ry1, rx0:rx1] = img_gray[ys:ye, xs:xe]
    return roi, x0, y0

def pad_zero(roi, padding):
    if padding <= 0:
        return roi
    return np.pad(roi, ((padding, padding), (padding, padding)),
                  mode="constant", constant_values=0)

def roi_to_inbuf(padded):
    return padded.reshape(-1).astype(np.uint8)
```

Após essas etapas, o vetor resultante deve possuir exatamente `TOTAL_PIXELS_IN` elementos. Esse valor é lido diretamente do hardware, garantindo que software e IP estejam sincronizados.

4.6 Passo 6: Configuração do IP via AXI-Lite (Start e Clear)

Antes de iniciar a transferência de dados, o IP precisa ser colocado em um estado conhecido. Neste projeto, isso é feito por dois registradores principais:

- `clear`: limpa estados internos e sinalizações do IP;
- `start`: inicia o processamento do descritor HOG.

O acesso ocorre via AXI-Lite utilizando as funções auxiliares:

```
def rd(a):
    return hog.read(a)

def wr(a, v):
    hog.write(a, int(v))
```

A sequência utilizada é:

```
wr(0x18, 0x1)  # clear
wr(0x00, 0x1)  # start
```

Essa ordem garante que o IP não herde estados de uma execução anterior e esteja pronto para consumir a nova ROI enviada via stream.

4.7 Passo 7: Transferência de Dados com AXI-DMA (MM2S e S2MM)

Com o vetor de entrada pronto e o IP configurado, o próximo passo é acionar o AXI-DMA para movimentar os dados.

O objetivo do DMA é:

- Enviar o vetor `in_buf` do PS para o IP HOG (canal MM2S);
- Receber o descritor `out_buf` do IP HOG de volta ao PS (canal S2MM).

No PYNQ, isso é feito utilizando `transfer()` e `wait()`.

Uma prática recomendada adotada neste projeto é iniciar primeiro a recepção (`recvchannel`) e somente depois iniciar o envio (`sendchannel`). Isso reduz a chance de deadlock, pois garante que o sistema esteja pronto para armazenar o resultado assim que o hardware começar a transmiti-lo.

```
dma.recvchannel.transfer(out_buf)
dma.sendchannel.transfer(in_buf)

dma.sendchannel.wait()
dma.recvchannel.wait()
```

Após a conclusão das duas chamadas `wait()`, tem-se a garantia de que:

- o hardware consumiu toda a ROI enviada;
- o descritor completo foi gravado no buffer de saída.

4.8 Passo 8: Leitura do Descritor HOG e Função de Execução Completa

Por fim, a saída do acelerador é lida diretamente do buffer de saída, que contém `NUM_WORDS` palavras de 32 bits (`uint32`).

O projeto utiliza a função `hog_hw()`, que encapsula toda a sequência descrita anteriormente. Entretanto, o entendimento dos passos anteriores é essencial para depuração e validação do sistema.

A função completa é:

```
def hog_hw(padded_img_u8):
    flat = roi_to_inbuf(padded_img_u8)
    if flat.size != TOTAL_PIXELS_IN:
        raise ValueError("Tamanho do input nao bate com
           TOTAL_PIXELS_IN")
```

```
    in_buf[:] = flat
    out_buf[:] = 0

    wr(0x18, 0x1)  # clear
    wr(0x00, 0x1)  # start

    dma.recvchannel.transfer(out_buf)
    dma.sendchannel.transfer(in_buf)
    dma.sendchannel.wait()
    dma.recvchannel.wait()

    return np.array(out_buf, dtype=np.uint32)
```

Como a saída é retornada como um vetor `uint32`, ela pode ser diretamente armazenada, comparada ou convertida para bytes para análises adicionais.

5 Conclusão

Este tutorial apresentou, de forma passo a passo, como utilizar o framework PYNQ na plataforma Kria KV260 para integrar o Processing System (PS) com a Programmable Logic (PL) e executar um acelerador HOG em hardware.

Foram abordados o carregamento do overlay, o uso de registradores via AXI-Lite, a preparação da ROI, a transferência de dados via AXI-DMA e um conjunto de validações práticas para confirmar o funcionamento e auxiliar no diagnóstico de falhas.